AI Powered Static Analysis (SAST) using Ollama

In this practice we will install local LLM using Ollama. We will also build a python script which does SAST on a python application.

Stages:

1. Installing Ollama and downloading qwen2.5-coder
2. Performing SAST using a python script

Stage A - Installing Ollama and qwen3 Model

Step 1 - Install Ollama Provider and Download qwen2.5-coder LLM

1. Browse the link https://ollama.com/download
2. Click on 'macOS' / 'Windows' and download the appropriate installer
3. Complete the installation of Ollama on your machine

Step 2 - Search for Ollama LLMs

1. Browse https://ollama.com/library
2. Search for 'qwen2.5-coder' using the search box
   OR browse directly tohttps://ollama.com/library/qwen2.5-coder
3. Locate the LLM qwen2.5-coder:0.5b in the library

Stage C - Start Ollama Provider & Run qwen2.5-coder:0.5b LLM

Note: For Windows machines, the Ollama Engine is typically already running as a service. The ollama serve command can be ignored if the server is already active.

1. Start the Ollama server - for windows machines 'ollama service' auto starts. But for mac you need to execute the '`ollama serve`' command.
2. Download the qwen2.5-coder:0.5b model (approximately 397MB)
   Open a new command prompt / terminal and execute:
   `ollama pull qwen2.5-coder:0.5b`
   This command will download the model if not already present locally.
3. Browse the Ollama Library for additional versions. Higher parameters promise better LLM respones.

Stage D - Access the Playground to Submit Prompts

1. Start the model for interactive prompting
   Open a new command prompt / terminal and execute:
   `ollama run qwen2.5-coder:0.5b`
   This loads the model and presents the >>> prompt, ready for input.
2. Test with a sample prompt
   Type an idea, for example:
   `How many programming languages are you aware of?`
   → Press Enter key to see the response
3. Exit the interactive prompt
   Press Ctrl+D to exit from the >>> prompt

Stage B - Performing SAST using a python script

This workshop guides you through understanding a Python-based Static Application Security Testing (SAST) scanner that leverages a local LLM (Ollama with qwen2.5-coder:0.5b) to analyze code for security vulnerabilities and generate structured reports.

Steps Summary:

1. Understanding the Configuration and Setup
2. Exploring the Prompt Engineering for Security Analysis
3. Implementing File Collection and Source Reading
4. Integrating with Ollama API
5. Parsing and Validating LLM Responses
6. Generating Console Reports with Severity Formatting
7. Exporting JSON Reports and Setting Exit Codes

Step 1: Understanding the Configuration and Setup

Problem Statement

The scanner needs to know which model to use, where the repository is located, which files to scan, and what patterns to skip. This section defines the core configuration that drives the entire scanning process.

Steps

a. Within VSCode create a file named ‘sast-scan.py’ within C:\labs\ai-devops\sample-python-repo folder
b. Copy the contents of ‘sast-scan’ from trainer shared link to sast-scan.py file
c. Save the sast-scan.py file
d. Review the Python file by following below steps

Step 1.1: Configuration Constants

```
OLLAMA_URL  = "http://localhost:11434/api/generate"
MODEL       = "qwen2.5-coder:0.5b"
REPO_ROOT   = Path(__file__).parent
SCAN_EXTS   = {".py"}
SKIP_DIRS   = {"__pycache__", ".git", ".venv", "venv", "node_modules"}

TARGET_FILES = [
    "app.py",
    "database/db_handler.py",
    "utils/auth.py",
]
```

This section sets up the environment for the SAST scanner. It defines where to find the Ollama API, which model to use, and what files to scan.

Code Element	Explanation
`OLLAMA_URL`	Endpoint where Ollama API is listening for generate requests
`MODEL`	Specifies the local LLM model (qwen2.5-coder:0.5b) for code analysis
`REPO_ROOT`	Dynamically sets the repository root to the script's parent directory
`SCAN_EXTS`	Set of file extensions to scan (currently only Python files)
`SKIP_DIRS`	Directories to ignore during scanning (caches, version control, virtual envs)
`TARGET_FILES`	Explicit list of relative file paths to scan within the repository

Step 2: Exploring the Prompt Engineering for Security Analysis

Problem Statement

The scanner must instruct the LLM to perform security-focused code review and return results in a structured, parseable format.

Steps

a. Continue working in [sast-scan.py]
b. Review the prompt template section

Step 2.1: SAST Prompt Template

```
SAST_PROMPT = """You are a security-focused code reviewer performing Static Application Security Testing
(SAST).

Analyze the following Python source file and identify ALL security vulnerabilities and code quality
issues.

For EACH issue found, respond ONLY in valid JSON using this exact schema:
{
  "file": "<filename>",
  "issues": [
    {
      "id": "SAST-001",
      "severity": "CRITICAL|HIGH|MEDIUM|LOW|INFO",
      "category": "<e.g. SQL Injection, Hardcoded Secret, Insecure Deserialization>",
      "line_range": "<e.g. 12-15 or 12>",
      "description": "<what the issue is>",
      "recommendation": "<how to fix it>"
    }
  ],
  "summary": {
    "total_issues": <int>,
    "critical": <int>,
    "high": <int>,
    "medium": <int>,
    "low": <int>
  }
}

Focus on:
- SQL Injection
- Hardcoded credentials / secrets
- Insecure deserialization (pickle, yaml.load)
- Weak cryptography / token generation
- Missing authentication or authorization
- Debug mode enabled in production
- Bare except clauses hiding errors
- Missing input validation
- Insecure network binding
- Plaintext password storage

File: {filename}
Source code:
```python
{source_code}
Respond ONLY with valid JSON. No markdown, no explanation outside the JSON."""
```

This prompt template instructs the LLM to act as a security reviewer and return results in a specific JSON format.
```

Step 3: Implementing File Collection and Source Reading

Problem Statement

The scanner needs to locate target files and prepare source code with line numbers for accurate issue reporting.

Steps

a. Continue working in `sast-scan.py`
b. Review the helper functions section

Step 3.1: File Collection Function

```
def collect_files(repo_root: Path, targets: list) -> list:
    """Return list of Path objects for target files that exist."""
    files = []
    for rel in targets:
        p = repo_root / rel
        if p.exists():
            files.append(p)
        else:
            print(f"  [WARN] File not found, skipping: {p}")
    return files
```

This function validates which target files actually exist in the repository.

Code Element	Explanation
Parameters	repo_root (Path object) and targets (list of relative file paths)
repo_root / rel	Constructs absolute path by joining repo root with relative target
p.exists()	Checks if the file actually exists on disk
Warning message	Prints warning for missing files without stopping execution
Return value	List of existing file Path objects for further processing

Step 3.2: Source Code Reading with Line Numbers

```
def read_source(filepath: Path) -> str:
    """Read source file with line numbers prepended (helps LLM cite line_range)."""
    lines = filepath.read_text(encoding="utf-8", errors="replace").splitlines()
    numbered = [f"{i+1:>4}: {line}" for i, line in enumerate(lines)]
    return "\n".join(numbered)
```

This function reads the source file and adds line numbers to help the LLM accurately cite issue locations.

Code Element	Explanation
read_text()	Reads entire file content with UTF-8 encoding, replacing unreadable characters
.splitlines()	Splits content into individual lines without newline characters
List comprehension	Creates formatted strings with line number (right-aligned, 4 spaces) followed by colon and the code line
f"{i+1:>4}"	Formats line number with 4-character width, right-aligned
Return value	Single string with all lines joined by newline characters

Step 4: Integrating with Ollama API

Problem Statement

The scanner must communicate with the local Ollama instance, send prompts, and handle connection errors.

Steps

a. Continue working in sast-scan.py
b. Review the Ollama integration functions

Step 4.1: Sending Prompts to Ollama

```
def ask_ollama(prompt: str) -> str:
    """Send prompt to local Ollama and return raw response text."""
    payload = {
        "model": MODEL,
        "prompt": prompt,
        "stream": False,
        "options": {
            "temperature": 0.1,
            "num_predict": 2048,
        }
    }
    try:
        resp = requests.post(OLLAMA_URL, json=payload, timeout=120)
        resp.raise_for_status()
        return resp.json().get("response", "")
    except requests.exceptions.ConnectionError:
        print("\n[ERROR] Cannot reach Ollama. Is it running? Try: ollama serve")
        sys.exit(1)
    except requests.exceptions.Timeout:
        print("\n[ERROR] Ollama timed out. Model may be loading -- retry.")
        sys.exit(1)
```

This function handles HTTP communication with Ollama and includes error handling.

Code Element	Explanation
payload dictionary	Constructs request body with model name, prompt, and generation options
temperature: 0.1	Low temperature ensures deterministic, consistent responses for security analysis
num_predict: 2048	Maximum tokens to generate (sufficient for detailed security reports)
requests.post()	Sends HTTP POST request to Ollama API with 120-second timeout
resp.raise_for_status()	Raises exception for HTTP error status codes
resp.json().get("response")	Extracts the generated text from Ollama's response
Exception handling	Catches connection and timeout errors, exits gracefully with guidance

Step 5: Parsing and Validating LLM Responses

Problem Statement

LLM responses may contain markdown formatting or malformed JSON. The scanner must extract valid JSON and provide fallback handling.

Steps

a. Continue working in sast-scan.py
b. Review the response parsing function

Step 5.1: JSON Response Parsing

```
def parse_json_response(raw: str, filename: str) -> dict:
    """Extract JSON from LLM response, handling markdown fences."""
    text = raw.strip()
```

```
    if text.startswith("```"):
        lines = text.splitlines()
        text = "\n".join(
            line for line in lines
            if not line.strip().startswith("```")
        ).strip()

    try:
        return json.loads(text)
    except json.JSONDecodeError:
        return {
            "file": filename,
            "issues": [{
                "id": "SAST-PARSE-ERR",
                "severity": "INFO",
                "category": "Parse Error",
                "line_range": "N/A",
                "description": "LLM response could not be parsed as JSON.",
                "recommendation": "Review raw output in sast_report.json under raw_responses.",
                "_raw": raw[:500]
            }],
            "summary": {"total_issues": 0, "critical": 0, "high": 0, "medium": 0, "low": 0}
        }
```

This function cleans LLM responses and attempts to parse JSON, with fallback for parsing failures.

Code Element	Explanation
raw.strip()	Removes leading/trailing whitespace from LLM response
Markdown fence detection	Checks if response starts with triple backticks (```) |
Filter lines	Removes lines that contain only triple backticks to extract pure JSON
json.loads(text)	Attempts to parse cleaned text as JSON
JSONDecodeError handler	Creates a synthetic error issue when JSON parsing fails
_raw field	Stores first 500 characters of raw response for debugging
Fallback summary	Returns empty summary structure to maintain data consistency

Step 6: Generating Console Reports with Severity Formatting

Problem Statement

Scan results need to be displayed in a human-readable format with color-coded severity levels for quick assessment.

Steps

a. Continue working in sast-scan.py
b. Review the report generation section

Step 6.1: Severity Configuration and Report Printing

```
SEVERITY_ORDER = {"CRITICAL": 0, "HIGH": 1, "MEDIUM": 2, "LOW": 3, "INFO": 4}
SEVERITY_COLOR = {
    "CRITICAL": "\033[91m",
    "HIGH":     "\033[33m",
    "MEDIUM":   "\033[93m",
    "LOW":      "\033[94m",
    "INFO":     "\033[37m",
}
RESET = "\033[0m"
BOLD  = "\033[1m"

def print_report(all_results: list):
    print("\n" + "=" * 70)
    print(f"{BOLD}  SAST SCAN REPORT -- {datetime.now().strftime('%Y-%m-%d %H:%M:%S')}{RESET}")
    print("=" * 70)
```

```
    for result in all_results:
        file_path = result.get("file", "unknown")
        issues    = result.get("issues", [])
        summary   = result.get("summary", {})

        sorted_issues = sorted(
            issues,
            key=lambda x: SEVERITY_ORDER.get(x.get("severity", "INFO"), 99)
        )

        for issue in sorted_issues:
            sev   = issue.get("severity", "INFO")
            color = SEVERITY_COLOR.get(sev, "")
            print(f"  {color}[{sev}]{RESET}  {issue.get('id', '???')} -- {issue.get('category', '')}")
            print(f"         Line   : {issue.get('line_range', '?')}")
            print(f"         Issue  : {issue.get('description', '')}")
            print(f"         Fix    : {issue.get('recommendation', '')}")
            print()
```

This section defines color codes and functions to print formatted reports.

Code Element	Explanation
SEVERITY_ORDER	Dictionary mapping severity strings to numeric sort order (lower = more severe)
SEVERITY_COLOR	ANSI escape codes for colored terminal output (red, yellow, blue, grey)
RESET	ANSI code to reset terminal color to default
BOLD	ANSI code for bold text formatting
sorted()	Sorts issues by severity using the custom order dictionary
lambda function	Extracts severity from each issue with default "INFO" and fallback value 99
Color application	Wraps severity label in appropriate color code then resets

Step 7: Exporting JSON Reports and Setting Exit Codes

Problem Statement

For CI/CD integration (Jenkins), the scanner must produce machine-readable output and return appropriate exit codes.

Steps

a. Continue working in sast-scan.py
b. Review the JSON export and exit code functions

Step 7.1: JSON Report Export

```
def save_json_report(all_results: list, output_path: Path):
    report = {
        "scan_timestamp": datetime.now().isoformat(),
        "model": MODEL,
        "repo": str(REPO_ROOT),
        "files_scanned": len(all_results),
        "results": all_results,
    }
    output_path.write_text(json.dumps(report, indent=2), encoding="utf-8")
    print(f"\n  [OK] JSON report saved -> {output_path}")
```

This function creates a comprehensive JSON report for downstream processing.

Code Element	Explanation
datetime.now().isoformat()	Creates ISO-8601 formatted timestamp for audit trail
files_scanned	Count of files processed (useful for validation)
results	Includes all scan results from each file
json.dumps(report, indent=2)	Converts dictionary to pretty-printed JSON with 2-space indentation
write_text()	Saves JSON to specified file path with UTF-8 encoding

Step 7.2: Exit Code Logic for CI/CD

```
def compute_exit_code(all_results: list) -> int:
    critical = sum(r.get("summary", {}).get("critical", 0) for r in all_results)
    high     = sum(r.get("summary", {}).get("high", 0)     for r in all_results)
    medium   = sum(r.get("summary", {}).get("medium", 0)   for r in all_results)
    low      = sum(r.get("summary", {}).get("low", 0)      for r in all_results)

    if critical > 0 or high > 0:
        return 1
    if medium > 0 or low > 0:
        return 2
    return 0
```

This function determines exit codes for Jenkins pipeline integration.

Code Element	Explanation
Generator expressions	Sums severity counts across all scanned files using nested dictionary access
Default 0	Uses .get() with default 0 to handle missing keys safely
Condition chain	Checks critical/high first (most severe) then medium/low
Return 1	Indicates build failure - critical or high severity issues found
Return 2	Indicates unstable build - only medium or low severity issues found
Return 0	Indicates successful build - no issues found

Step 8: Execute

Execute the Python file by following these steps:

1. Launch a new command prompt / terminal
2. Ensure Ollama is running:
   `ollama serve`
3. Pull the required model:
   `ollama pull qwen2.5-coder:0.5b`
4. Install required Python dependency:
   ```
   cd c:\labs\ai-devops
   c:\labs\ai-devops>.venv\Scripts\activate.bat
   (.venv) c:\labs\ai-devops>pip install requests
   ```
5. Run the SAST scanner:
   ```
   cd c:\labs\ai-devops\sample-python-repo
   python sast-scan.py
   ```

Key Takeaways:

- Prompt Engineering Matters: Crafting precise prompts with JSON schema constraints ensures predictable, parseable LLM outputs
- Error Handling is Critical: Always implement fallbacks for LLM responses that may not conform to expected formats
- CI/CD Integration: Exit codes and machine-readable JSON reports enable seamless pipeline integration
- Line Number Preparation: Pre-formatting source code with line numbers significantly improves LLM's ability to accurately reference issue locations
- Severity-Based Sorting: Implementing severity ordering allows security teams to prioritize critical issues first